

INFORME TÉCNICO: SISTEMA DE DESTINOS, PAQUETES Y RESERVAS PARA VIAJES AVENTURA

Evaluación Sumativa 4 — Desarrollo de software orientado a objetos y seguro

Asignatura: Programación Orientada a Objeto Seguro (TI3021)

■ **Sección:** 2026/P TI3V21/V-IEI-N2-P3-C1/V Valparaíso IEI

Nombre del académico: Rubén Alejandro Schnettler Lucero

Nombre del estudiante: Claudio Medina M.

Fecha de entrega: 5 de octubre de 2026

Índice

I. Introducción.....	3
II. Objetivos.....	3
III. Desarrollo.....	4
3.1 Análisis del caso y supuestos.....	4
3.2 Requerimientos funcionales y no funcionales.....	4
3.3 Priorización y verificación de los requerimientos.....	6
3.4 Modelado de procesos (BPMN).....	7
3.5 Diagrama de casos de uso.....	7
3.6 Diagrama de clases UML.....	8
3.7 Trazabilidad entre requerimientos y modelos.....	9
3.8 Planificación ágil con Scrum.....	10
3.9 Implementación: clases, persistencia y CRUD.....	11
3.10 Seguridad: autenticación y protección de datos.....	13
3.11 Auditoría de seguridad y evidencia de ejecución.....	13
3.12 Uso crítico de inteligencia artificial.....	14
IV. Conclusiones.....	15
V. Referencias bibliográficas.....	15

I. Introducción

Viajes Aventura es una agencia de turismo de tres socios que arma paquetes combinando destinos nacionales. Hoy toda su operación depende de una planilla compartida y de un cuaderno de reservas. El caso entregado en la asignatura muestra las consecuencias de esa forma de trabajar durante la última temporada: destinos repetidos con nombres distintos, precios calculados a mano con errores, **6 reservas por sobre el cupo**, **3 reservas sobre fechas ya vencidas**, **19 reservas duplicadas** y datos personales de los clientes (RUT y teléfono) circulando por correo.

Este informe documenta el desarrollo de un **sistema de terminal en Python** que reemplaza esa planilla. Los administradores gestionan destinos y arman paquetes de 2 a 5 destinos, cuyo precio calcula el propio sistema; los clientes se registran, reservan, consultan su historial y cancelan. La información queda guardada en una base de datos **SQLite**, las contraseñas se protegen con **hash y sal**, el acceso se controla por rol y los datos sensibles nunca se muestran completos.

El trabajo siguió el ciclo completo de un proyecto de software: análisis del caso y requerimientos, modelado con BPMN y UML, planificación con Scrum, implementación orientada a objetos, auditoría de seguridad y revisión crítica del uso de inteligencia artificial. Todo el código y las evidencias están publicados en el repositorio github.com/claudiomedina01-debug/poo-viajes-aventura, que este informe cita en cada sección.

II. Objetivos

Objetivo General:

Desarrollar un sistema orientado a objetos, seguro y con persistencia en base de datos, que permita a Viajes Aventura gestionar destinos, paquetes y reservas, resolviendo los problemas detectados en el caso y documentando cada decisión de diseño, seguridad y uso de inteligencia artificial.

Objetivos Específicos:

1. Identificar, clasificar, priorizar y redactar los requerimientos funcionales y no funcionales a partir de las reglas de negocio y los problemas del caso.
2. Modelar los procesos y la estructura del sistema con diagramas BPMN, de casos de uso y de clases UML, asegurando la trazabilidad entre requerimientos y modelos.
3. Planificar el desarrollo con el marco Scrum, definiendo roles, Product Backlog, Sprint Backlog, tiempos y entregables.
4. Implementar en Python las clases del modelo UML con persistencia en SQLite y operaciones CRUD sobre destinos, paquetes y reservas.
5. Proteger las credenciales y los datos sensibles con librerías oficiales de Python, y evaluar la seguridad del sistema mediante pruebas y una auditoría con apoyo de IA.

III. Desarrollo

3.1 Análisis del caso y supuestos

El análisis partió de las entrevistas a los tres socios (Paulina Ovalle, catálogo; Matías Bórquez, reservas; Ignacio Salas, administración) y de las cifras de la temporada. Cada problema se registró con su evidencia para que todo requerimiento posterior pueda justificarse con un dato del caso.

Tabla 1. Problemas detectados en el caso

ID	Problema	Evidencia del caso
P1	Catálogo desordenado: destinos repetidos y destinos que ya no se operan siguen visibles	4 destinos no operados visibles
P2	Precios calculados a mano y que cambian después de vender	Error "de un cero"; 4 paquetes con precio distinto al cobrado
P3	Se venden más lugares que el cupo	6 reservas sobre el cupo
P4	Se venden paquetes con fecha de salida vencida	3 reservas
P5	Reservas duplicadas, anotadas por socios distintos	19 duplicadas
P6	RUT y teléfono en una planilla que circula por correo	Entrevista a Ignacio
P7	No hay historial por cliente	31 consultas respondidas a mano
P8	No hay acceso controlado: cualquiera edita todo	3 socios editan la misma planilla

El caso pide explícitamente declarar un supuesto cuando falta información. Se adoptaron seis, cada uno con su fundamento:

Tabla 2. Supuestos adoptados

ID	Vacío del caso	Supuesto adoptado
S1	¿Qué pasa si un cliente desiste?	El cliente cancela su reserva antes de la salida; la reserva no se borra, pasa a "cancelada" y libera el cupo
S2	¿Qué pasa al terminar la temporada?	Un paquete vencido deja de mostrarse como disponible, pero no se borra
S3	¿Quién modifica el catálogo?	Solo el Administrador ; el primero se crea al usar el sistema por primera vez
S4	El pago está fuera del alcance	La reserva queda "confirmada"; la transferencia se verifica a mano
S5	¿Cuándo se "publica" un paquete (R7)?	Al crearlo; en ese momento queda fijo su precio
S6	¿Cómo evitar duplicados?	Un cliente no puede tener 2 reservas activas del mismo paquete

3.2 Requerimientos funcionales y no funcionales

Se tradujeron las 17 reglas de negocio (R1–R17) y los supuestos en **15 requerimientos funcionales** y **12 no funcionales**. Todos usan la misma estructura: un identificador, la frase "El sistema debe...", el origen (regla o supuesto), el problema que resuelven y un **criterio de aceptación** verificable. Por ejemplo:

RF-12. El sistema debe permitir que un cliente autenticado reserve un paquete indicando la cantidad de personas. **Origen:** R11, R12, R14–R16, S6. **Problema:** P3, P4, P5. **Criterio de aceptación:** se rechaza si supera el cupo disponible, si la salida ya pasó, si las personas son menos de 1 o si el cliente ya tiene una reserva activa de ese paquete.

Tabla 3. *Requerimientos funcionales*

ID	El sistema debe...	Origen	Prioridad
RF-01	Registrar clientes con nombre, RUT, correo, teléfono y contraseña	R9	Alta
RF-02	Permitir iniciar sesión con correo y contraseña	R11	Alta
RF-03	Permitir cerrar la sesión	R11	Media
RF-04	Crear el primer administrador la primera vez que se ejecuta	S3	Alta
RF-05	Registrar destinos (nombre único, zona, descripción, duración, costo > 0)	R1, R2	Alta
RF-06	Modificar destinos sin alterar paquetes ya publicados	R1, R2, R7	Media
RF-07	Eliminar un destino sin paquetes o marcarlo "no disponible"	R8	Alta
RF-08	Listar los destinos indicando su disponibilidad	R8	Media
RF-09	Crear paquetes de 2 a 5 destinos distintos, con fechas y cupo válidos	R3–R5	Alta
RF-10	Calcular el precio (costos + margen) y fijarlo al publicar	R6, R7, S5	Alta
RF-11	Listar paquetes vigentes con su cupo disponible	R14, R15, S2	Alta
RF-12	Permitir reservar a un cliente autenticado	R11–R16, S6	Alta
RF-13	Calcular el total (precio × personas) y guardarlo sin cambios	R13	Alta
RF-14	Mostrar al cliente el historial de sus reservas	R11, R12	Alta
RF-15	Permitir cancelar la reserva propia antes de la salida	S1	Media

Tabla 4. *Requerimientos no funcionales*

ID	Tipo	El sistema debe...	Prioridad
RNF-01	Seguridad	Guardar las contraseñas como hash con sal (hashlib.scrypt), nunca como texto	Alta
RNF-02	Seguridad	Bloquear la cuenta tras 3 intentos fallidos seguidos	Alta
RNF-03	Seguridad	Exigir claves de 8 caracteres o más y dar un mensaje de error genérico	Media
RNF-04	Control de acceso	Verificar el rol antes de cada operación, en el menú y en las clases	Alta
RNF-05	Privacidad	No mostrar el RUT ni el teléfono en listados ni errores	Alta
RNF-06	Privacidad	Mantener la base de datos fuera del repositorio	Media
RNF-07	Validación	Validar todo dato: correo, RUT (módulo 11), teléfono, fechas y números	Alta
RNF-08	Integridad	Reforzar las reglas en la base con NOT NULL, UNIQUE, CHECK y claves foráneas	Media
RNF-09	Seguridad	Usar consultas parametrizadas en todo el SQL	Alta
RNF-10	Confiabilidad	No cerrarse ante un error ni mostrar rutas o trazas técnicas	Alta
RNF-11	Usabilidad	Ofrecer un menú en español con mensajes claros	Baja

ID	Tipo	El sistema debe...	Prioridad
RNF-12	Mantenibilidad	Usar solo la librería estándar y separar dominio (viajes.py) y menú (main.py)	Media

3.3 Priorización y verificación de los requerimientos

La prioridad se asignó con tres niveles y una **pregunta clave** para cada uno, de modo que la decisión sea repetible y no dependa de la intuición:

Tabla 5. Criterio de priorización

Prioridad	Criterio	Pregunta clave
Alta	Lo exige una regla del caso o el alcance, o resuelve un problema grave medido (sobreventa, datos expuestos, precios)	¿El sistema sirve sin esto? → No
Media	Mejora importante, pero el sistema funciona sin ella, o nace de un supuesto propio	¿Funciona sin esto? → Sí, pero peor
Baja	Comodidad o presentación	¿Alguien lo extrañaría en la primera versión? → Poco

El resultado fue **18 requerimientos de prioridad Alta, 8 Media y 1 Baja**. Los de prioridad Alta se construyeron primero (Sprints 2 y 3).

Para comprobar que los requerimientos responden al problema real del cliente, se verificó en tres direcciones: **cada problema** P1–P8 tiene al menos un requerimiento que lo resuelve, **cada regla** R1–R17 está cubierta y **cada punto del alcance** acordado por los socios tiene su requerimiento, sin agregar nada fuera de él (pagos, facturación, app móvil o correos quedaron excluidos).

Tabla 6. Verificación: problema del caso → requerimiento que lo resuelve

Problema	Cifra	Requerimientos	Cómo se resuelve
P1 Catálogo desordenado	4 destinos	RF-05, RF-07, RF-08, RNF-08	Nombre único; destino en uso pasa a "no disponible"
P2 Precios mal calculados	4 paquetes	RF-10, RF-13, RF-06	El sistema calcula el precio y lo fija; el total no cambia
P3 Sobreventa	6 reservas	RF-11, RF-12	Se calcula el cupo disponible y se rechaza el exceso
P4 Fechas vencidas	3 reservas	RF-11, RF-12	Un paquete vencido no se lista ni se reserva
P5 Duplicados	19 reservas	RF-12, RNF-07	Una sola reserva activa por cliente y paquete (S6)
P6 Datos expuestos	Planilla por correo	RNF-01, 05, 06, 09	Hash, enmascarado, base fuera del repositorio, SQL seguro
P7 Sin historial	31 consultas	RF-01, RF-14, RF-15	Cada cliente ve su historial; cancelar no borra
P8 Sin control de acceso	3 socios	RF-02, RF-04, RNF-02, 04	Login, roles y bloqueo por intentos

3.4 Modelado de procesos (BPMN)

Se modelaron los dos procesos centrales del negocio con notación **BPMN 2.0**. Cada diagrama usa carriles (lanes) para separar quién hace cada tarea, y compuertas exclusivas (♦) para las decisiones en que el sistema valida y puede rechazar.

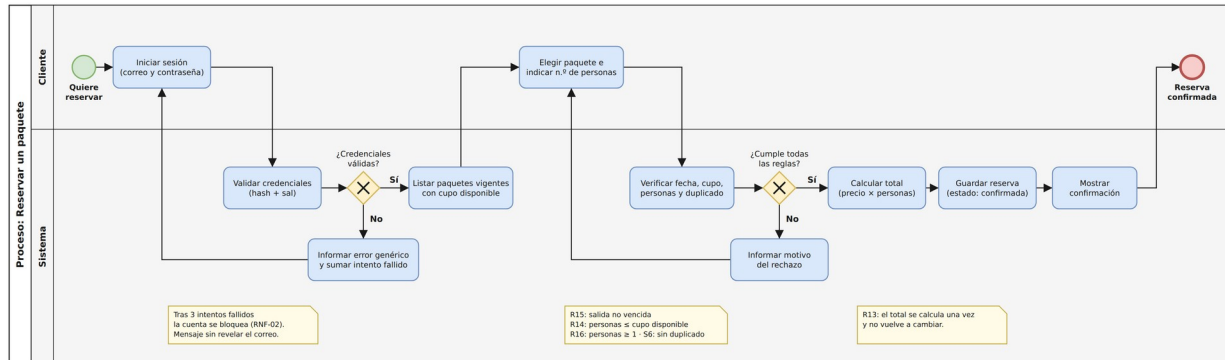


Figura 1. Proceso BPMN "Reservar paquete" (carriles Cliente y Sistema).

En **Reservar paquete**, el cliente inicia sesión; el sistema valida las credenciales (con bloqueo al tercer intento fallido), lista solo los paquetes vigentes con cupo y, cuando el cliente elige uno, verifica fecha, cupo, cantidad de personas y duplicado antes de calcular el total y guardar la reserva. Cualquier validación fallida devuelve un mensaje y no guarda nada.

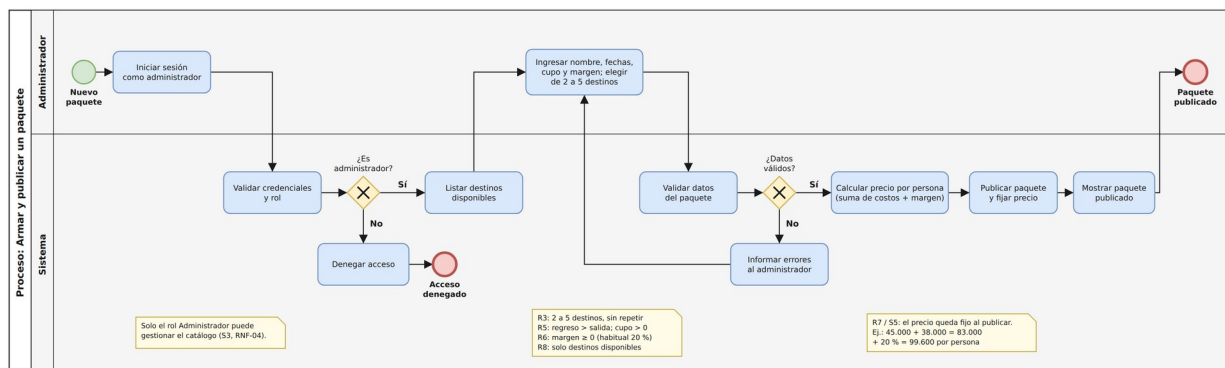


Figura 2. Proceso BPMN "Armar paquete" (carriles Administrador y Sistema).

En **Armar paquete**, el sistema valida que el usuario tenga rol de administrador, lista los destinos disponibles y, con los datos ingresados, comprueba las reglas R3 y R5 (2 a 5 destinos sin repetir, regreso posterior a la salida, cupo mayor que cero). Solo entonces calcula el precio con el margen (R6) y publica el paquete, dejando el precio fijo (R7).

3.5 Diagrama de casos de uso

El diagrama de casos de uso muestra los **15 casos de uso** del sistema y sus dos actores. Ambos actores heredan de un actor **Usuario**, que concentra lo que comparten (iniciar y cerrar sesión). La relación «include» indica pasos obligatorios que se reutilizan: crear un paquete siempre incluye calcular su precio, y reservar siempre incluye verificar el cupo.

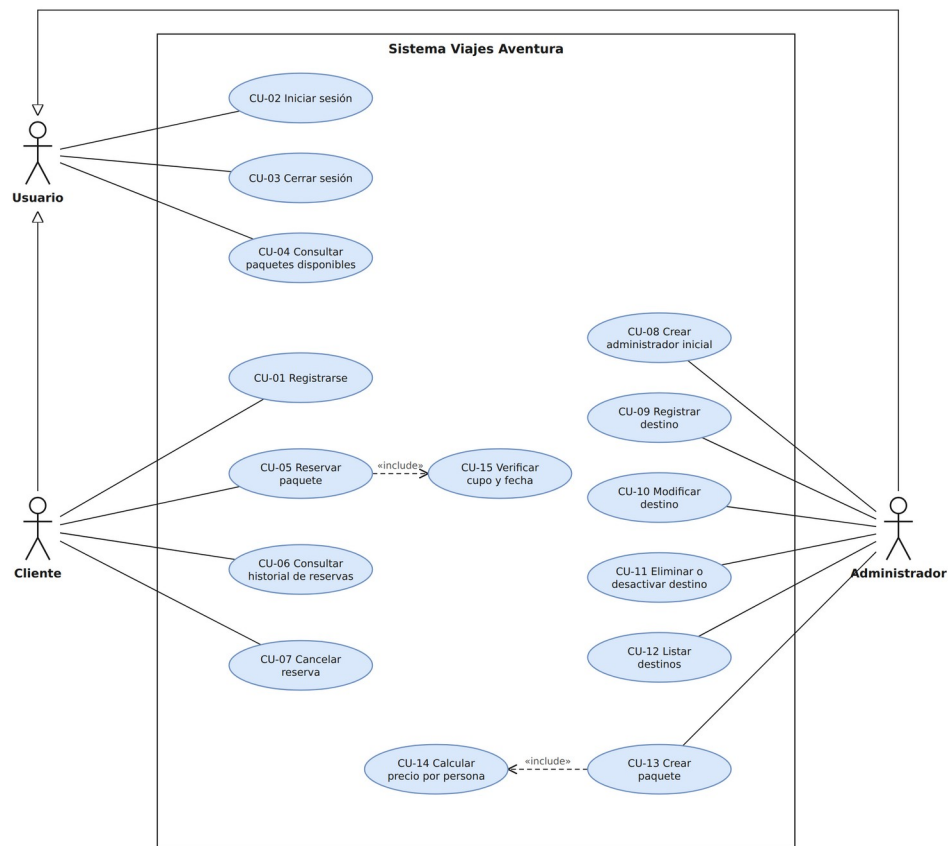


Figura 3. Diagrama de casos de uso de Viajes Aventura.

3.6 Diagrama de clases UML

El diagrama de clases define **8 clases** y es la referencia directa del código: cada atributo y método del diagrama existe en `viajes.py` con el mismo nombre. Se actualizó dos veces durante el proyecto para mantener esa correspondencia (al programar y al corregir el hallazgo H2), y la versión inicial se conserva en `diagramas/historico`.

Tabla 7. Relaciones del modelo y su significado

Relación	Entre	Multiplicidad	Por qué
Herencia	Cliente y Administrador → Usuario	—	Comparten correo, clave protegida y bloqueo; Usuario es abstracta y cada hijo implementa <code>es_admin()</code> a su manera (polimorfismo)
Composición (◆ negro)	SistemaViajes → BaseDatos	1 a 1	El sistema crea su propia base de datos y no existe sin ella
Agregación (◇ blanco)	Paquete → Destino	0..* a 2..5	Un paquete agrupa de 2 a 5 destinos (R3), pero los destinos existen solos y pueden estar en varios paquetes (R4)
Asociación	Reserva → Cliente / Paquete	* a 1	Cada reserva pertenece a un cliente y a un paquete; ambos pueden tener muchas reservas

Relación	Entre	Multiplicidad	Por qué
Asociación	SistemaViajes → Usuario	0..1	Sesión activa: como máximo un usuario conectado

El **encapsulamiento** se aplica en los datos sensibles: `_hash_clave`, `_sal`, `_intentos_fallidos`, `_rut` y `_telefono` son privados (prefijo `_`), y desde fuera solo se accede a ellos a través de métodos como `verificar_clave()` o `rut_enmascarado()`.

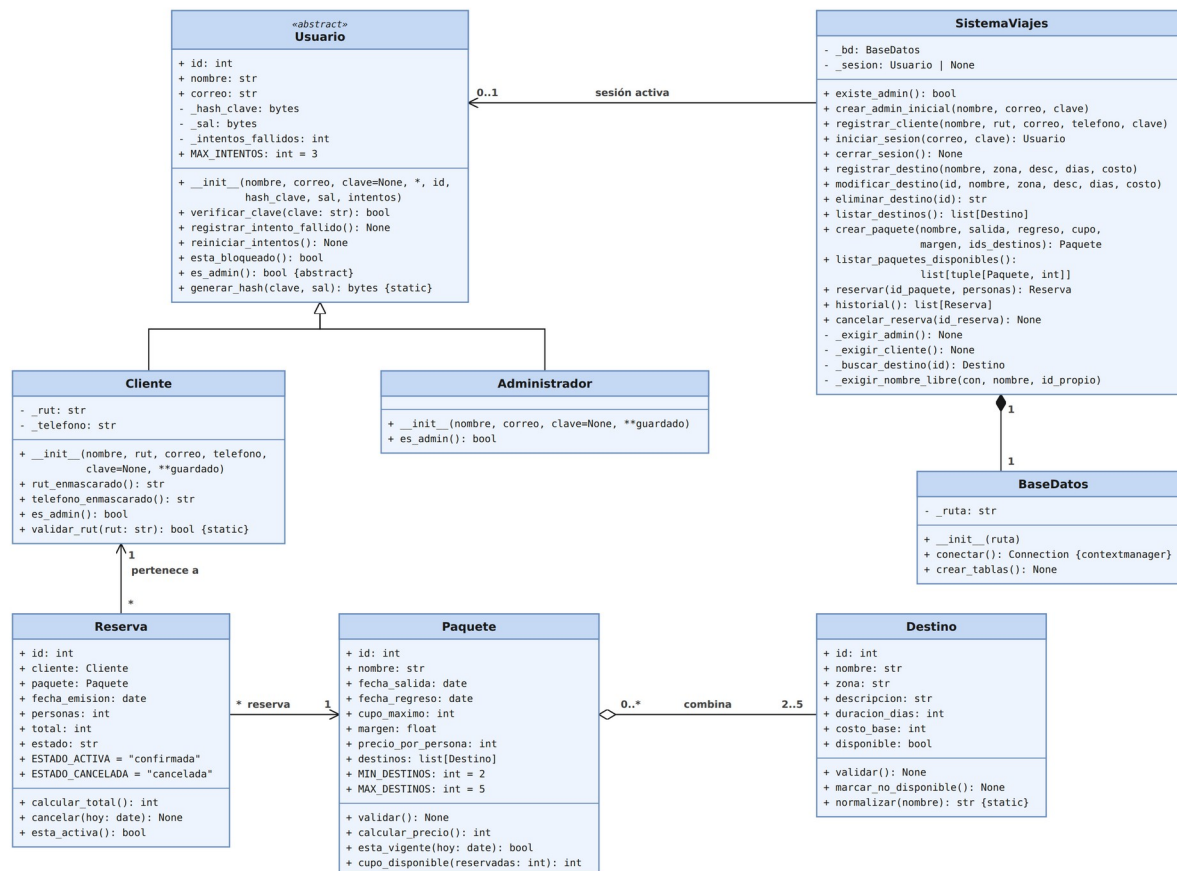


Figura 4. Diagrama de clases UML (versión vigente, igual al código).

3.7 Trazabilidad entre requerimientos y modelos

La matriz de trazabilidad (docs/TRAZABILIDAD.md) sigue cada requerimiento desde el **problema del caso** hasta la **clase y el método** que lo cumplen, y termina en la **evidencia de ejecución** que lo demuestra. A continuación se presenta un extracto con los requerimientos funcionales; la matriz completa incluye también los 12 no funcionales y el paso BPMN de cada uno.

Tabla 8. Extracto de la matriz de trazabilidad (ST = bloque de SALIDA_TERMINAL.md)

RF	Problema · Regla	Caso de uso	Clase · método	Evidencia
RF-01	P6, P7 · R9	CU-01	SistemaViajes.registrar_cliente() · Cliente.validar_rut()	ST 2.5, 2.6, 5.1 · pruebas RF-01, H1

RF	Problema · Regla	Caso de uso	Clase · método	Evidencia
RF-02	P8 · R11	CU-02	SistemaViajes.iniciar_sesion() · Usuario.verificar_clave()	ST 2.1, 2.6 · prueba RNF-01/02/03/09
RF-04	P8 · S3	CU-08	SistemaViajes.crear_admin_inicial()	ST 2.1 · prueba RF-04
RF-05	P1 · R1, R2	CU-09	SistemaViajes.registrar_destino() · Destino.normalizar()	ST 2.2, 2.3, 5.2 · pruebas RF-05, H2
RF-07	P1 · R8	CU-11	SistemaViajes.eliminar_destino() · Destino.marcar_no_disponible()	Prueba RF-07 / R8
RF-09	P1, P2 · R3–R5	CU-13	SistemaViajes.crear_paquete() · Paquete.validar()	ST 2.4 · prueba RF-09/10
RF-10	P2 · R6, R7	CU-14	Paquete.calcular_precio()	ST 2.4 (\$99.600) · prueba RF-06/R7
RF-11	P3, P4 · R14, R15	CU-04	listar_paquetes_disponibles() · Paquete.esta_vigente()	ST 2.7, 2.8 · prueba RF-11/R15
RF-12	P3–P5 · R11–R16	CU-05, CU-15	SistemaViajes.reservar() · Paquete.cupo_disponible()	ST 2.7, 2.8 · prueba RF-12/13
RF-13	P2 · R13	CU-05	Reserva.calcular_total()	ST 2.8 (\$199.200)
RF-14	P7 · R11	CU-06	SistemaViajes.historial() · _exigir_cliente()	ST 2.8 · pruebas RF-14/15, RNF-04
RF-15	P3, P7 · S1	CU-07	SistemaViajes.cancelar_reserva() · Reserva.cancelar()	Prueba RF-14/15

La revisión de coherencia confirmó que **los 15 RF tienen caso de uso**, que **todo caso de uso viene de un RF**, que los dos procesos BPMN solo usan pasos que existen como requerimiento, que **las 8 clases participan** en algún requerimiento y que **los 27 requerimientos tienen evidencia de ejecución**.

3.8 Planificación ágil con Scrum

El proyecto se organizó con **Scrum**. Como el trabajo es individual, una misma persona cumplió los tres roles; para no mezclarlos, cada rol tiene su momento y su artefacto: primero se planifica y prioriza (Product Owner), luego se organiza el sprint (Scrum Master) y al final se construye y prueba (equipo de desarrollo).

Tabla 9. Roles Scrum

Rol	Responsabilidad	Artefacto
Product Owner	Representar a los socios, traducir sus necesidades en requerimientos, priorizar y aceptar cada entrega	Product Backlog · REQUERIMIENTOS.md
Scrum Master / Líder	Planificar los sprints, cuidar los plazos, eliminar bloqueos y registrar retrospectivas	Sprint Backlog · registro de avance
Equipo de desarrollo	Diseñar, programar, probar y documentar	Código, diagramas y docs/

El **Product Backlog** tiene **13 historias de usuario** con el formato "Como *rol*, quiero *acción*, para *beneficio*", estimadas en puntos de historia (escala 1, 2, 3, 5): **39 puntos** en total, 11 de prioridad Alta y 2 Media. Algunas de ellas:

Tabla 10. *Extracto del Product Backlog*

ID	Historia de usuario	Prioridad	Puntos
HU-02	Como usuario , quiero iniciar y cerrar sesión de forma segura, para que nadie más use mi cuenta	Alta	5
HU-07	Como administrador , quiero armar un paquete con 2 a 5 destinos y que el precio se calcule solo, para no equivocarme en la suma	Alta	5
HU-08	Como cliente , quiero ver solo los paquetes vigentes y con cupo, para no elegir uno que no puedo comprar	Alta	3
HU-09	Como cliente , quiero reservar indicando cuántas personas viajan, para asegurar mi lugar	Alta	5
HU-11	Como cliente , quiero cancelar mi reserva antes de la salida, para liberar el cupo si no puedo viajar	Media	2
HU-13	Como equipo , quiero auditar la seguridad y documentar el uso de IA, para demostrar que el código es confiable	Alta	3

Tabla 11. *Sprint Backlog y cronograma (30 de septiembre al 5 de octubre de 2026)*

Sprint	Fechas	Objetivo y contenido	Entregable	Estado
0 · Análisis	30 sep – 1 oct	Leer la pauta, crear el repositorio, requerimientos y supuestos	REQUERIMIENTOS.md	Terminado
1 · Diseño	1 – 2 oct	Casos de uso, 2 BPMN, diagrama de clases, trazabilidad, planificación	diagramas/ + TRAZABILIDAD.md	Terminado
2 · Catálogo	2 – 3 oct	Base SQLite y lado administrador (HU-01, 04–07, 12): 16 puntos	viajes.py + main.py	Terminado
3 · Clientes y seguridad	3 – 4 oct	Lado cliente y seguridad (HU-02, 03, 08–11, 13): 23 puntos	Login, reservas, AUDITORIA, evidencia	Terminado
4 · Cierre	4 – 5 oct	README, informe, revisión contra la rúbrica, guion de defensa	Entrega en el AAI	En curso

Una historia se consideró **terminada** (Definition of Done) solo si cumplía sus criterios de aceptación, coincidía con el diagrama de clases, tenía su ejecución registrada en docs/SALIDA_TERMINAL.md, su uso de IA anotado en docs/ANALISIS_IA.md y estaba subida a GitHub con un commit descriptivo. Las ceremonias se adaptaron al trabajo individual (planificación de 15 minutos, revisión diaria del avance y retrospectiva de tres preguntas al cierre de cada sprint). Por ejemplo, la retrospectiva del Sprint 2 detectó que el diagrama de clases había quedado desactualizado al programar, y se adoptó la regla de actualizarlo junto con el código.

3.9 Implementación: clases, persistencia y CRUD

El código se separó en dos archivos según su responsabilidad: `viajes.py` contiene las 8 clases, la base de datos, el CRUD, el login y las pruebas; `main.py` es solo el menú y **no contiene SQL**.

Se usó únicamente la **librería estándar** de Python (sqlite3, hashlib, secrets, re, abc, datetime, unicodedata), por lo que el sistema no requiere instalar dependencias.

Tabla 12. Clases y tablas de la base de datos

Clase	Qué representa	Tabla SQLite
Usuario (abstracta)	Persona con cuenta: nombre, correo, clave protegida, intentos fallidos	usuarios
Cliente / Administrador	Los dos roles; el cliente agrega RUT y teléfono privados	usuarios (columna rol)
Destino	Lugar con zona, duración, costo base y disponibilidad	destinos
Paquete	Combina 2 a 5 destinos, con fechas, cupo, margen y precio fijo	paquetes + paquete_destino
Reserva	Cliente + paquete + personas + total fijo + estado	reservas
BaseDatos	Abre la conexión, maneja transacciones y crea las tablas	—
SistemaViajes	Coordina la sesión, los permisos y todas las operaciones	—

Persistencia. Las 5 tablas se relacionan con **claves foráneas**: paquete_destino resuelve la relación de muchos a muchos entre paquetes y destinos, con ON DELETE RESTRICT sobre el destino para que no se pueda borrar uno que está en uso (R8). Cada operación abre y cierra su propia conexión dentro de una **transacción** (BEGIN IMMEDIATE), que se confirma si todo sale bien y se revierte (ROLLBACK) ante cualquier error.

CRUD. Sobre los destinos se implementa el ciclo completo: crear (registrar_destino), leer (listar_destinos), actualizar (modificar_destino) y eliminar o desactivar (eliminar_destino), que borra el destino si no está en ningún paquete y, si lo está, lo marca "no disponible". Los paquetes se crean y listan; las reservas se crean, se listan en el historial y se cancelan cambiando su estado, sin borrarse (S1).

Validación en tres capas. Un dato inválido se detiene en la primera capa que lo detecte: (1) el **menú** revisa el formato mientras el usuario escribe (números, fechas dd-mm-aaaa, RUT); (2) las **clases** revisan las reglas del negocio con sus métodos validar(); y (3) la **base de datos** rechaza lo que se salte las dos anteriores, mediante restricciones NOT NULL, UNIQUE, CHECK y claves foráneas. Por ejemplo:

```
CREATE TABLE IF NOT EXISTS destinos (
    nombre      TEXT      NOT NULL UNIQUE COLLATE NOCASE,
    costo_base  INTEGER NOT NULL CHECK (costo_base > 0),      -- R2
    ...)
CREATE UNIQUE INDEX ux_reserva_activa ON reservas (cliente_id, paquete_id)
WHERE estado = 'confirmada';      -- S6: sin duplicados
```

Manejo de errores. Cada error esperado del usuario se lanza como ValueError o PermissionError con un mensaje claro, y el menú lo captura con try/except y vuelve a ofrecer la opción. Los errores técnicos de la base (sqlite3.Error) se capturan al nivel más alto con un mensaje genérico, sin mostrar rutas ni trazas (RNF-10).

Pruebas automáticas. Ejecutar `python viajes.py` corre **15 pruebas** que usan una base temporal y cubren las reglas del caso, los requerimientos de seguridad y las correcciones de la auditoría. Todas terminan en "OK" (ver sección 3.11).

3.10 Seguridad: autenticación y protección de datos

La seguridad se diseñó desde el inicio con **librerías oficiales de Python**, sin implementar algoritmos propios:

- **Contraseñas con hash y sal (R10, RNF-01).** Cada clave se transforma con `hashlib.scrypt`, una función lenta a propósito que encarece adivinar claves por fuerza bruta, usando una **sal aleatoria de 16 bytes** generada con `secrets.token_bytes`. Así, dos usuarios con la misma clave tienen hashes distintos y en la base nunca aparece una contraseña legible.
- **Verificación segura.** Al iniciar sesión se recalcula el hash y se compara con `secrets.compare_digest`, que tarda lo mismo sin importar dónde difieran los valores (evita ataques por tiempo de respuesta).
- **Validación de credenciales (RNF-02, RNF-03).** La cuenta se **bloquea al tercer intento fallido** y el mensaje es siempre "Correo o contraseña incorrectos", para no revelar qué correos existen.
- **Control de acceso (RNF-04).** El rol se verifica **dentro de las clases** (`_exigir_admin()`, `_exigir_cliente()`), no solo en el menú: aunque alguien llame directamente al método, un cliente no puede gestionar el catálogo ni ver reservas ajenas.
- **Datos sensibles (R17, RNF-05, RNF-06).** El RUT y el teléfono son atributos privados y solo se muestran enmascarados (`12.***.***-*, *****5678`). El archivo de la base está excluido del repositorio en `.gitignore`.
- **Inyección SQL (RNF-09).** Todas las consultas usan parámetros `?`; escribir `' OR 1=1 --` como correo no permite entrar.

```
def generar_hash(clave, sal):          # Usuario
    return hashlib.scrypt(clave.encode("utf-8"), salt=sal, n=2**14, r=8, p=1, dklen=64)

def verificar_clave(self, clave):
    calculado = Usuario.generar_hash(clave, self._sal)
    return secrets.compare_digest(calculado, self._hash_clave)
```

3.11 Auditoría de seguridad y evidencia de ejecución

La seguridad se evaluó en tres etapas: **revisión del código** contra los RNF, **pruebas automáticas** y una **demostración manual** en la que se usó el sistema como lo haría una persona real, provocando errores a propósito. El resultado se documentó en `docs/AUDITORIA.md`: se verificaron **14 controles** (todos cumplen) y se encontraron **4 hallazgos**.

Tabla 13. Hallazgos de la auditoría

#	Severidad	Hallazgo	Corrección	Estado
H1	Media	El nombre aceptó 12345678-5 (un RUT sin letras)	Patrón que exige letras, en la clase Usuario (vale para ambos roles por herencia)	Corregido

#	Severidad	Hallazgo	Corrección	Estado
H2	Media	"Cajon del Maipo" y "Cajón del Maipo" se guardaban como destinos distintos: volvía el problema P1	Destino.normalizar() ignora tildes, mayúsculas y espacios dobles al comparar	Corregido
H3	Baja	El RUT inválido se avisaba al final del registro	pedir_rut() lo valida en el menú apenas se escribe	Corregido
H4	Baja	Tras corregir H1, el nombre inválido también se avisa al final	Validar el nombre en el menú (mejora documentada)	Pendiente

Los hallazgos H1 a H3 **no los detectaban las pruebas automáticas**; aparecieron solo al probar como un usuario real. Después de corregirlos se agregaron dos pruebas nuevas (las pruebas pasaron de 13 a **15 de 15**) y se repitió la demostración para comprobar cada arreglo:

```
Nombre completo: 12345678
RUT (12345678-5): 11111111-2
  x RUT inválido: revise el dígito verificador (ejemplo: 12345678-5).      ← H3
...
  x El nombre solo puede tener letras y espacios (2 a 60 caracteres).      ← H1
Nombre: cajón del Maipo
  x Ya existe un destino con ese nombre: Cajon del Maipo.                  ← H2
```

La auditoría también registra con honestidad los **riesgos que quedan**: el RUT y el teléfono se guardan sin cifrar dentro del archivo local, una cuenta bloqueada no se puede desbloquear desde el menú, no existe una bitácora de acciones y la clave solo exige 8 caracteres. Cada uno tiene una mejora propuesta para una versión futura.

Apoyo de IA en la auditoría. Claude apoyó la revisión del código y propuso las correcciones de H1 a H3, pero ninguna se aceptó sin comprobarla: primero con las pruebas automáticas en el computador del estudiante y después repitiendo a mano los errores de la demostración original. Además, al revisar esa nueva demostración se detectó que la corrección de H1 propuesta por la IA rechazaba bien el nombre, pero no consideraba **cuándo** se avisa el error; por eso se registró como hallazgo H4 en vez de darlo por cerrado.

3.12 Uso crítico de inteligencia artificial

Se usaron tres herramientas de IA: **Claude** (Anthropic) como tutor y apoyo de diseño, **Claude Code** para revisar la organización de carpetas y **GitHub Copilot**, que sugiere mensajes de commit. La regla fue registrar cada propuesta **en el momento** en docs/ANALISIS_IA.md y no usar ninguna sin comprobarla contra el caso, la rúbrica o las pruebas.

Tabla 14. Resumen del uso crítico de IA (12 registros)

Decisión	N.º	Ejemplo real
Adoptar	4	Las correcciones de H1–H3 se adoptaron solo después de comprobarlas con 15 pruebas en el computador propio y una nueva demostración manual
Modificar	6	El código del bloqueo propuesto por Claude avisaba el error dentro de la transacción: el ROLLBACK deshacía el contador de intentos y la cuenta nunca se bloqueaba. La prueba lo detectó y se corrigió guardando el contador antes de avisar

Decisión	N.º	Ejemplo real
Modificar		Claude propuso priorizar con MoSCoW; se cambió a Alta/Media/Baja con una pregunta clave por nivel, más simple de justificar
Modificar		Copilot sugirió para la auditoría el mensaje "Revise y actualice... se mejoró la estructura", falso porque el documento estaba vacío; se reescribió
Descartar	2	Claude Code marcó como "faltantes" la guía y la rúbrica, que existían con otro nombre (falso positivo), y afirmó que un archivo tenía "la tabla llena" cuando estaba vacío

La conclusión de este registro es que la IA aceleró el trabajo, pero **se equivocó o necesitó ajustes en 8 de 12 casos**. Las pruebas automáticas, la demostración manual y la lectura atenta de cada propuesta fueron lo que permitió detectar esos errores antes de entregarlos.

IV. Conclusiones

Conclusión Claudio Medina M.: El sistema desarrollado resuelve los ocho problemas detectados en Viajes Aventura: el catálogo ya no admite destinos repetidos, el precio lo calcula el sistema y queda fijo, no se puede vender sobre el cupo ni sobre fechas vencidas, no hay reservas duplicadas, cada cliente ve su propio historial y los datos sensibles están protegidos. Cada una de estas soluciones se puede seguir, gracias a la matriz de trazabilidad, desde el problema del caso hasta la línea de código y la prueba que la demuestra.

El aprendizaje más importante fue que **las pruebas automáticas no bastan**. Las 13 pruebas iniciales pasaban, pero al usar el sistema como una persona real aparecieron tres hallazgos; uno de ellos (H2) hacía volver exactamente el problema del caso que el sistema debía resolver. También comprobé que el diagrama de clases es un documento vivo: tuve que actualizarlo dos veces para que siguiera siendo igual al código.

Sobre la inteligencia artificial, concluyo que es una herramienta muy útil para explicar conceptos y proponer soluciones, pero no reemplaza la revisión. En 8 de 12 casos hubo que corregir o descartar lo que propuso, incluido un error de seguridad real en el bloqueo de cuentas. Mi rol fue verificar cada sugerencia con evidencia antes de aceptarla.

Como mejoras futuras quedan cifrar los datos personales dentro de la base, permitir que el administrador desbloquee cuentas dejando registro, agregar una bitácora de acciones y completar la validación del nombre en el menú (H4).

V. Referencias bibliográficas

Anthropic. (2026). *Claude* [Modelo de lenguaje de gran tamaño]. <https://claude.ai>

GitHub. (2026). *GitHub Copilot* [Herramienta de inteligencia artificial].
<https://github.com/features/copilot>

INACAP. (2026a). *Caso Viajes Aventura: Unidad 4* [Material de la asignatura TI3021 Programación Orientada a Objeto Seguro]. Ambiente de Aprendizaje INACAP.

INACAP. (2026b). *Guía y rúbrica de la Evaluación Sumativa 4, Unidad 4* [Material de la asignatura TI3021]. Ambiente de Aprendizaje INACAP.

- Medina, C. (2026). *poo-viajes-aventura* [Repositorio de código fuente]. GitHub.
<https://github.com/claudiomedina01-debug/poo-viajes-aventura>
- Object Management Group. (2013). *Business Process Model and Notation (BPMN)* (Versión 2.0.2). <https://www.omg.org/spec/BPMN/2.0.2/>
- Object Management Group. (2017). *OMG Unified Modeling Language (UML)* (Versión 2.5.1). <https://www.omg.org/spec/UML/2.5.1/>
- OWASP Foundation. (s.f.-a). *Password storage cheat sheet*. OWASP Cheat Sheet Series.
https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- OWASP Foundation. (s.f.-b). *SQL injection prevention cheat sheet*. OWASP Cheat Sheet Series.
https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- Python Software Foundation. (s.f.-a). *hashlib — Secure hashes and message digests*. Python 3 Documentation. <https://docs.python.org/3/library/hashlib.html>
- Python Software Foundation. (s.f.-b). *secrets — Generate secure random numbers for managing secrets*. Python 3 Documentation.
<https://docs.python.org/3/library/secrets.html>
- Python Software Foundation. (s.f.-c). *sqlite3 — DB-API 2.0 interface for SQLite databases*. Python 3 Documentation. <https://docs.python.org/3/library/sqlite3.html>
- Schwaber, K., & Sutherland, J. (2020). *La Guía Scrum: La guía definitiva de Scrum: Las reglas del juego*. <https://scrumguides.org/>